EPI Educational Group

Smart Cat GPS Tracker

Étude Comparative des Architectures Logicielles



Présenté par : Salma Mlika
Encadré par : Madame Ismail Imen

Année universitaire 2025 - 2026

Plan

01 Introduction & Problématique

02 Présentation du Projet

03 Les 4 Architectures Étudiées

04 Discussion Comparative

05 Conclusion

Introduction & Problématique





🐱

❓

⚠️

Les chats disparaissent souvent plusieurs heures

Propriétaires inquiets : où est mon chat ?

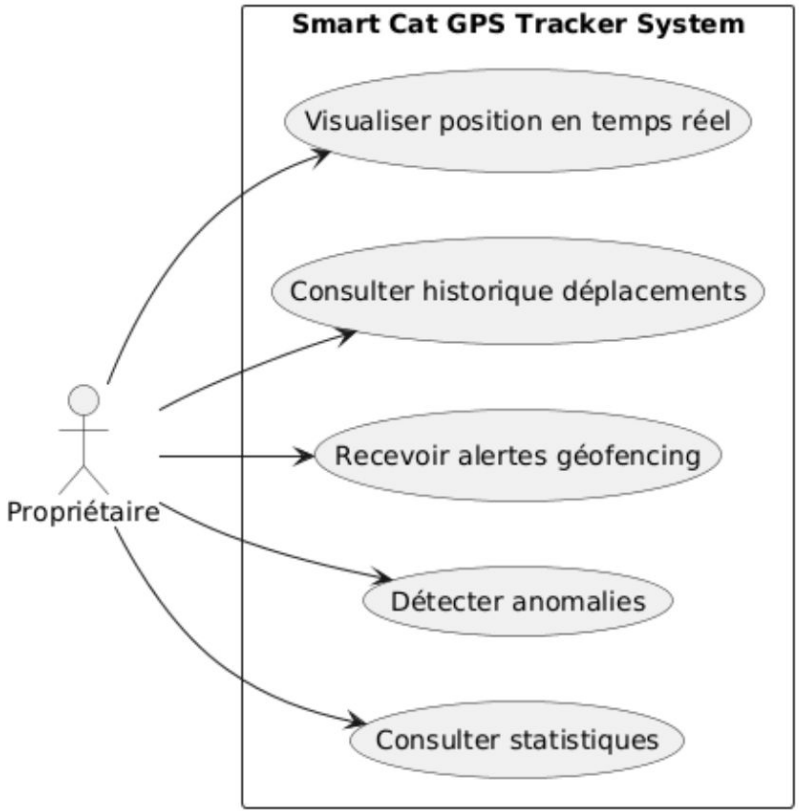
Zones dangereuses, comportements anormaux

💡 **Solution** : Système de tracking GPS intelligent en temps réel



Présentation du Projet

LE SYSTÈME



Besoins fonctionnels du système:

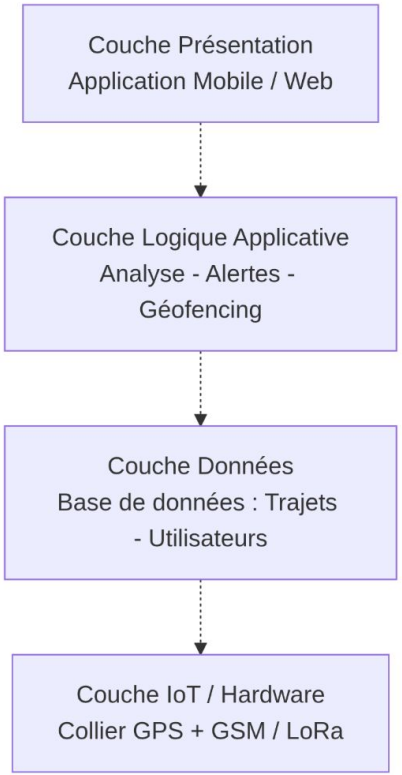
- ✅ Localisation temps réel
- ✅ Historique des trajets
- ✅ Zones de sécurité (Géofencing)
- ✅ Alertes intelligentes
- ✅ Analyses comportementales

Besoins Non Fonctionnels du Système

- ✅ Performance
- ✅ disponibilite
- ✅ Sécurité
- ✅ Scalabilité

Les 4 Architectures Étudiées

ARCHITECTURE 1 - MULTICOUCHES



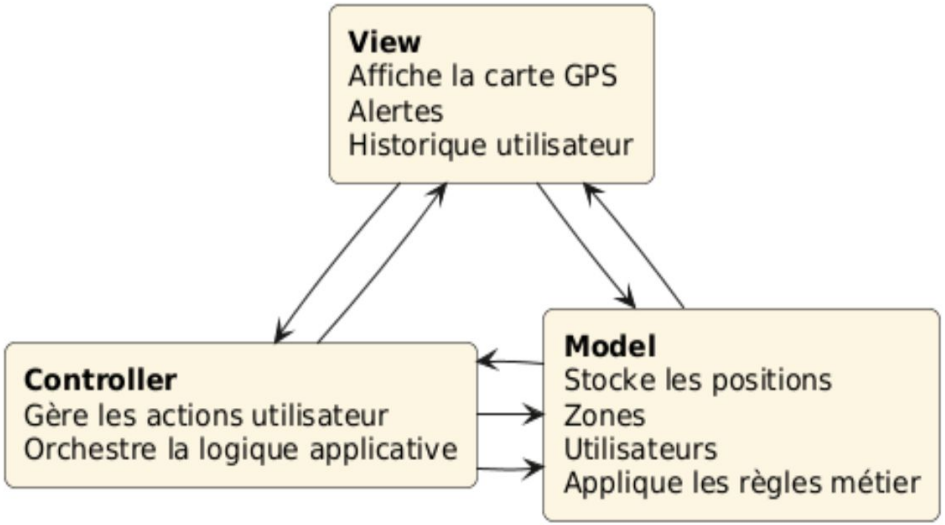
PRINCIPE : Organisation du système en couches horizontales, chaque couche ayant une responsabilité spécifique et communiquant uniquement avec les couches adjacentes.

✅ **Scénario : Mise à jour position GPS**

1. Collier GPS capture position → Couche Hardware
2. Couche Données stocke dans base
3. Couche Logique vérifie règles (géofencing)
4. Couche Présentation affiche sur carte

Les 4 Architectures Étudiées

ARCHITECTURE 2 - MVC



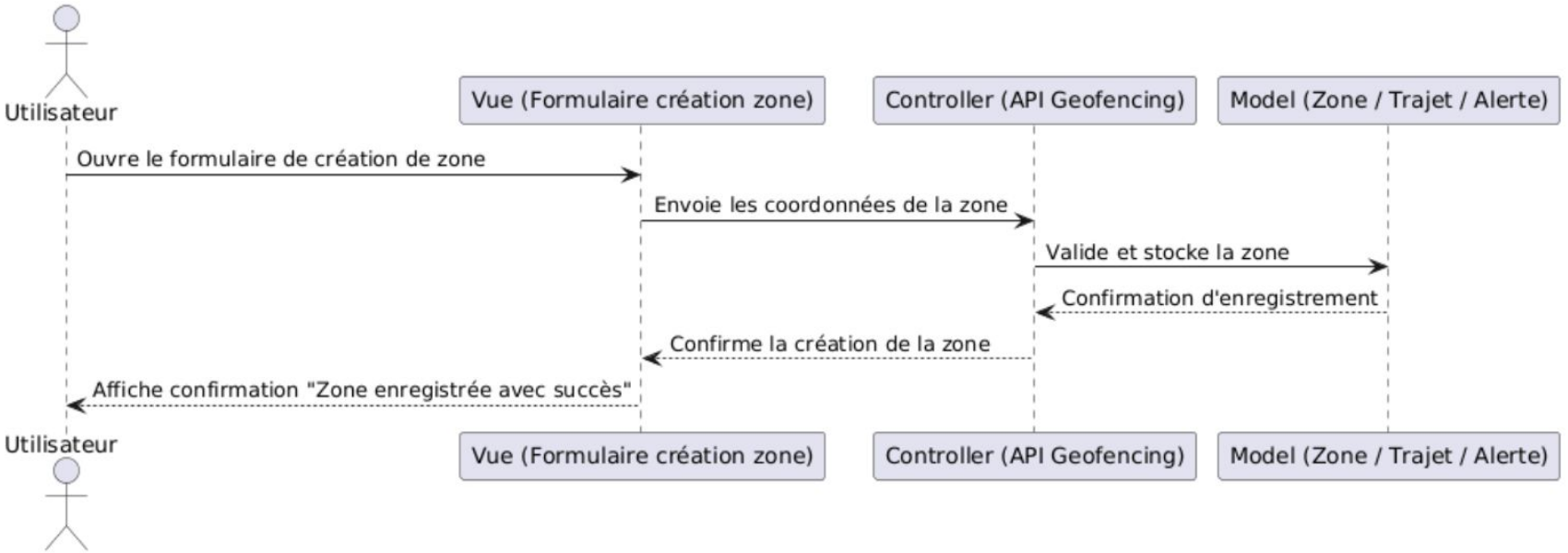
PRINCIPE : Séparation de l'application en trois composants interconnectés, chacun gérant un aspect spécifique : données, interface, et logique de contrôle.

MODEL (Modèle) — Représente les données ,Entités : User, Cat..)

VIEW (Vue) — Interface utilisateur

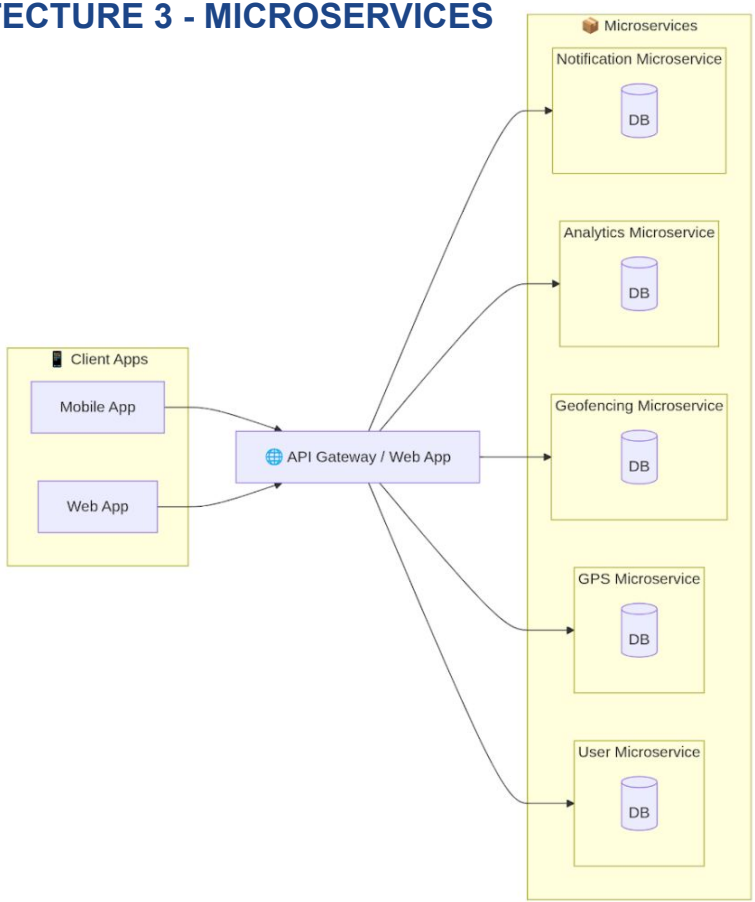
CONTROLLER (Contrôleur) Intermédiaire (Gestion des requêtes utilisateur..)

Diagramme de séquence : Création d’une zone géographique



Les 4 Architectures Étudiées

ARCHITECTURE 3 - MICROSERVICES



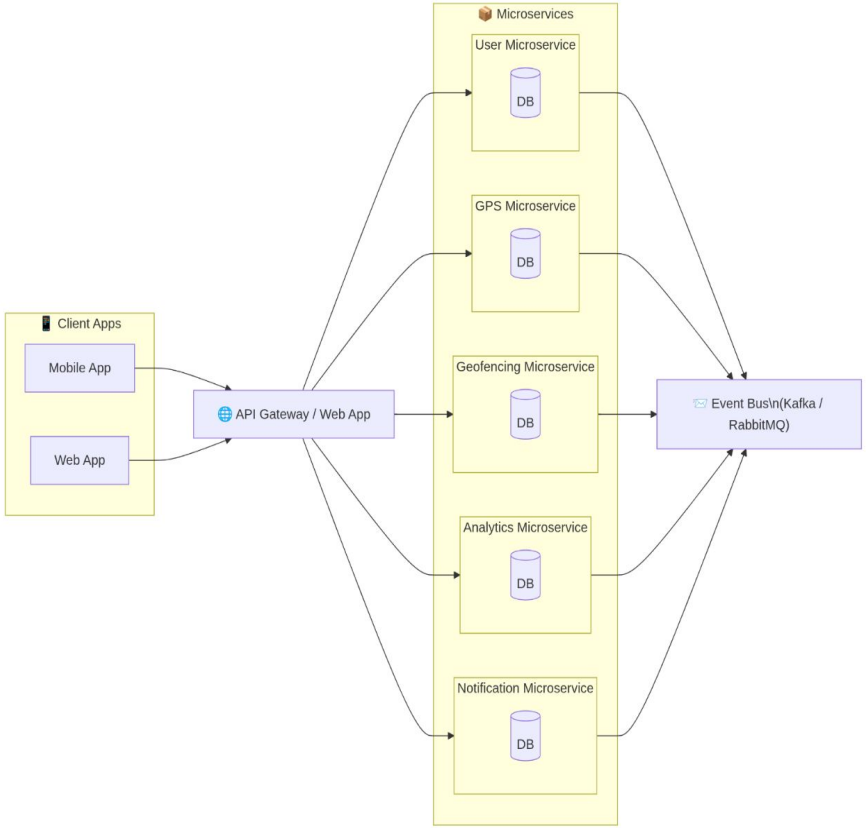
PRINCIPE : Décomposition du système en services autonomes, chacun responsable d'une fonction métier spécifique, déployable et scalable indépendamment.

✅ Scénario : Consultation de l'historique des trajets

1. L'utilisateur sélectionne "Historique" dans l'application.
2. Le **Frontend** envoie une requête au **API Gateway**.
3. Le **API Gateway** route la requête vers le **GPS Service**.
4. Le **GPS Service** lit l'historique depuis **gps_db**.
5. Les données sont renvoyées au **Frontend** sous forme de liste ou graphique.

Les 4 Architectures Étudiées

ARCHITECTURE 4 - EVENT-DRIVEN



PRINCIPE : Le système réagit aux événements (changement d'état significatif)en temps réel. Les composants communiquent via un bus d'événements central (Kafka), sans se connaître directement.

✅ **Scénario : Chat sort de sa zone de sécurité**

1. 📡 **Collier GPS → envoie position (36.80, 10.18)**
2. 💾 **GPS Service → publie "Location.Update" sur Kafka**
3. 🗺️ **Geofencing Service → détecte sortie de zone**
4. 🔔 **Geofencing → publie "Geofence.Breach** ↓
5. 📱 **Alert Service → envoie notification push** ↓
6. ✅ **Propriétaire reçoit "Minou est sorti du jardin !"**

⏱️ **Temps total : < 2 secondes**

Discussion Comparative

AVANTAGES ET INCONVÉNIENTS DES 4 ARCHITECTURES

	🏢 MULTICOUCHES	🎯 MVC	🔷 MICROSERVICES	⚡ EVENT-DRIVEN
✅ AVANTAGES	• Simple • Rapide • Faible coût	• Séparation claire • Dev parallèle • Bon pour UI	• Scalable • Résilient • Équipes autonomes	• Temps réel • Découplage • Parfait IoT
❌ INCONVÉNIENTS	• Monolithique • Pas scalable • Déploiement global	• Insuffisant IoT • Monolithique	• Complexe • Coût élevé • Debugging dur	• Très complexe • Kafka • Cohérence diff.
⭐ SCALABILITÉ	⭐⭐	⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐⭐
⏱️ TEMPS RÉEL	⭐⭐	⭐⭐	⭐⭐⭐⭐	⭐⭐⭐⭐⭐
💰 COÛT	Faible	Faible	Élevé	Élevé

Discussion Comparative

QUELLE ARCHITECTURE CHOISIR ?

CONTEXTE	ARCHITECTURE	POURQUOI ?
🧪 PROTOTYPE < 1000 users	MULTICOUCHES	• Rapide 2-3 mois • Faible coût
📱 APP WEB Focus UI/UX	MVC	• Interface riche • Frameworks
🏢 PRODUCTION Moyenne échelle	MICROSERVICES	• 10K-100K users • Haute dispo
🌐 IoT TEMPS RÉEL Grande échelle	MICROSERVICES + EVENT-DRIVEN	• 100K+ devices • < 2 sec latence

Discussion Comparative

ARCHITECTURE RECOMMANDÉE

```
╔═══════════════════════════════════════════════════════╗
║          RECOMMANDATION : ARCHITECTURE HYBRIDE        ║
║             MICROSERVICES + EVENT-DRIVEN              ║
╚═══════════════════════════════════════════════════════╝


POURQUOI ?
✓ 100 000+ colliers GPS actifs simultanément
✓ Positions GPS en flux continu temps réel
✓ Alertes critiques < 2 secondes (sortie zone, batterie)
✓ Scalabilité horizontale par service
✓ Résilience : disponibilité 99.9%
✓ Évolution : ajout IA/ML sans refonte

ARCHITECTURE CONCRÈTE :
- Frontend        → MVC (React Native + React.js)
- API Gateway     → Auth + Routage
- Services        → User, GPS, Geofencing, Alert, Analytics
- Communication   → Kafka (événements asynchrones)
- Données         → Database per Service
- Infra           → Kubernetes + Docker
```

APPROCHE PROGRESSIVE

```
PHASE 1 (3-6 mois) : MVP
├─ Architecture : MULTICOUCHES
├─ Users : 500
├─ Coût : 50K€
└─ Objectif : Valider le concept

        ↓ Migration

PHASE 2 (6-12 mois) : Lancement
├─ Architecture : MICROSERVICES
├─ Users : 10 000
├─ Coût : 200K€
└─ Objectif : Production stable

        ↓ Scaling

PHASE 3 (12+ mois) : Scale
├─ Architecture : MICROSERVICES + EVENT-DRIVEN
├─ Users : 100 000+
├─ Coût : 500K€+
└─ Objectif : Millions d'événements/jour + IA
```

Conclusion

📊 CE QU'ON A VU

- 4 architectures comparées
- Forces et faiblesses identifiées
- Contextes d'utilisation définis

🎯 RECOMMANDATION

MICROSERVICES + EVENT-DRIVEN

Pour système IoT temps réel grande échelle

✓ 100K+ devices
✓ Latence < 2 sec
✓ Disponibilité 99.9%
✓ Évolution continue



Le choix d'architecture est une décision stratégique qui impacte performance, coûts et évolutivité .

Pour un système IoT critique : investir dans une architecture robuste dès le départ = économies long terme



Merci pour votre attention

CREDITS: Diese Präsentationsvorlage wurde von **Slidesgo** erstellt, inklusive Icons von **Flaticon** und Infografiken & Bilder von **Freepik**